

CS 301
High-Performance Computing

Lab Report-05
Parallel Insertion and Deletion in Mover

Parth Karena (202301472)
Pranav Bavadiya (202301458)

March 31, 2026

Contents

1	Introduction	2
2	Hardware Details	2
2.1	LAB207 PCs	2
2.2	HPC Cluster	2
3	Algorithm Overview	3
3.1	Deferred Insertion Approach	3
3.2	Immediate Replacement Approach	3
4	Experiment 01: Scaling of Execution Time	3
4.1	Setup	3
4.2	Results	4
5	Experiment 02: Parallel Implementation and Scalability	7
5.1	Setup	7
6	Results	8
6.1	Lab PC vs. HPC Cluster Comparison	13
6.2	OpenMP Scalability and Effect of Insertion/Deletion	13
7	Bonus: Data Structures for Scalability	14
8	Conclusion	14

1 Introduction

This report presents the implementation and performance analysis of the extended Mover algorithm with dynamic particle insertion and deletion, as described in Assignment 05. Building on the scatter interpolation and Mover work from Assignments 03 and 04, we introduce two mechanisms: *delete particle* (removing particles that exit the 1×1 domain) and *insert particle* (randomly re-inserting replacement particles). Both a **Deferred Insertion Approach** and an **Immediate Replacement Approach** are implemented and compared. The Mover is subsequently parallelized using OpenMP and its scalability is studied.

2 Hardware Details

2.1 LAB207 PCs

- **Architecture:** x86_64
- **CPU(s):** 12
- **Threads per core:** 2
- **Cores per socket:** 6
- **Processor:** Intel Core i5-12500
- **L1 Cache:** 288 KB
- **L2 Cache:** 7.5 MB
- **L3 Cache:** 18 MB

2.2 HPC Cluster

- **Architecture:** x86_64
- **CPU(s):** 16
- **Threads per core:** 1
- **Cores per socket:** 8
- **Sockets:** 2
- **Processor:** Intel Xeon E5-2640 v3
- **L1 Cache:** 32 KB
- **L2 Cache:** 256 KB
- **L3 Cache:** 20480 KB

3 Algorithm Overview

3.1 Deferred Insertion Approach

1. Move all particles, applying random displacement $x \leftarrow x + r_x$, $y \leftarrow y + r_y$ where $r_x \in [-\Delta x, \Delta x]$, $r_y \in [-\Delta y, \Delta y]$.
2. Delete particles that exit the domain $[0, 1]^2$; increment a deletion counter.
3. Push vacant memory locations (voids) to the end of the data structure.
4. After all particles are processed, insert new particles at random positions inside the domain into the vacated memory slots.

3.2 Immediate Replacement Approach

1. Move each particle sequentially.
2. If the particle exits the domain, immediately replace it at the same memory location with a new particle at a random position inside the domain.

4 Experiment 01: Scaling of Execution Time

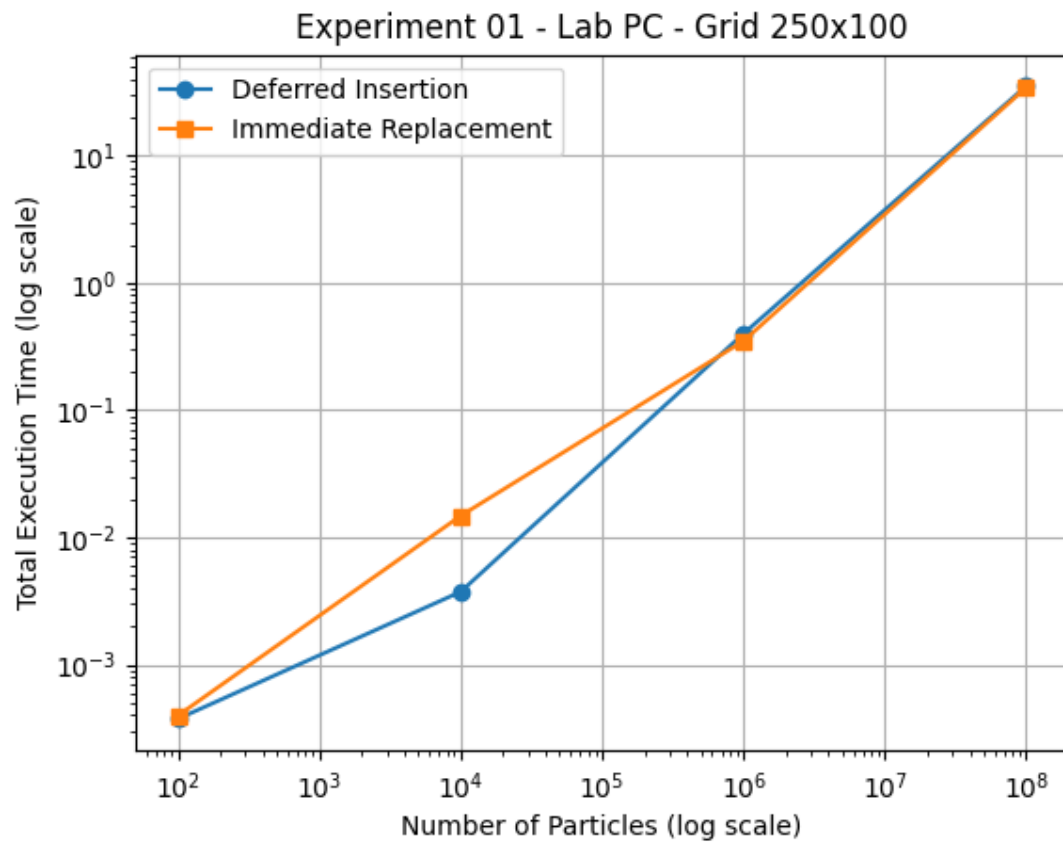
4.1 Setup

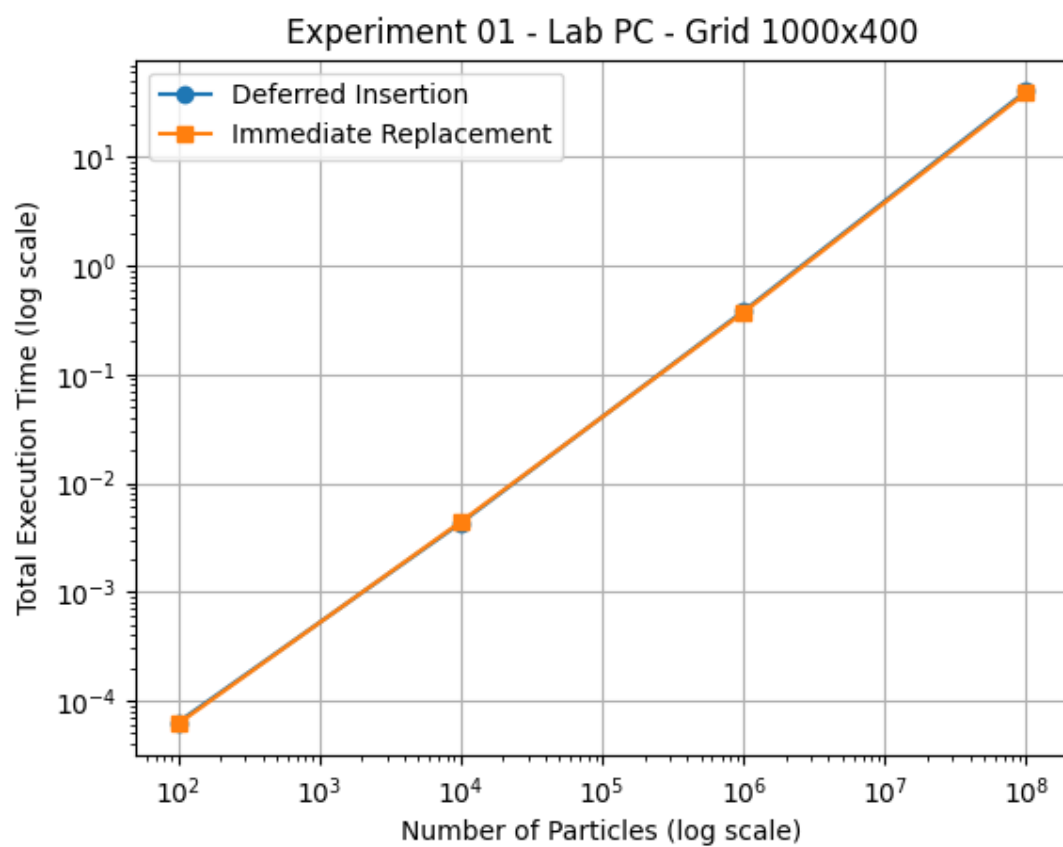
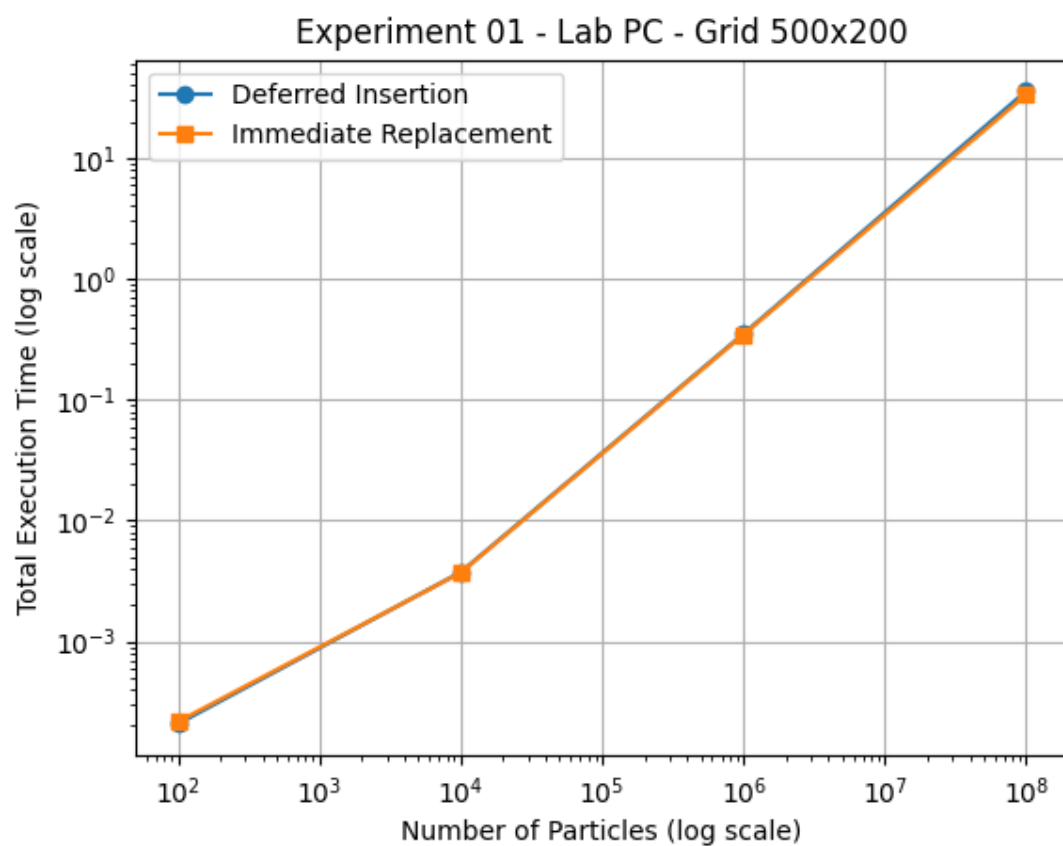
Three grid configurations were tested with particle counts $N_p \in \{10^2, 10^4, 10^6, 10^8, 10^9\}$ and $\text{Maxiter} = 10$:

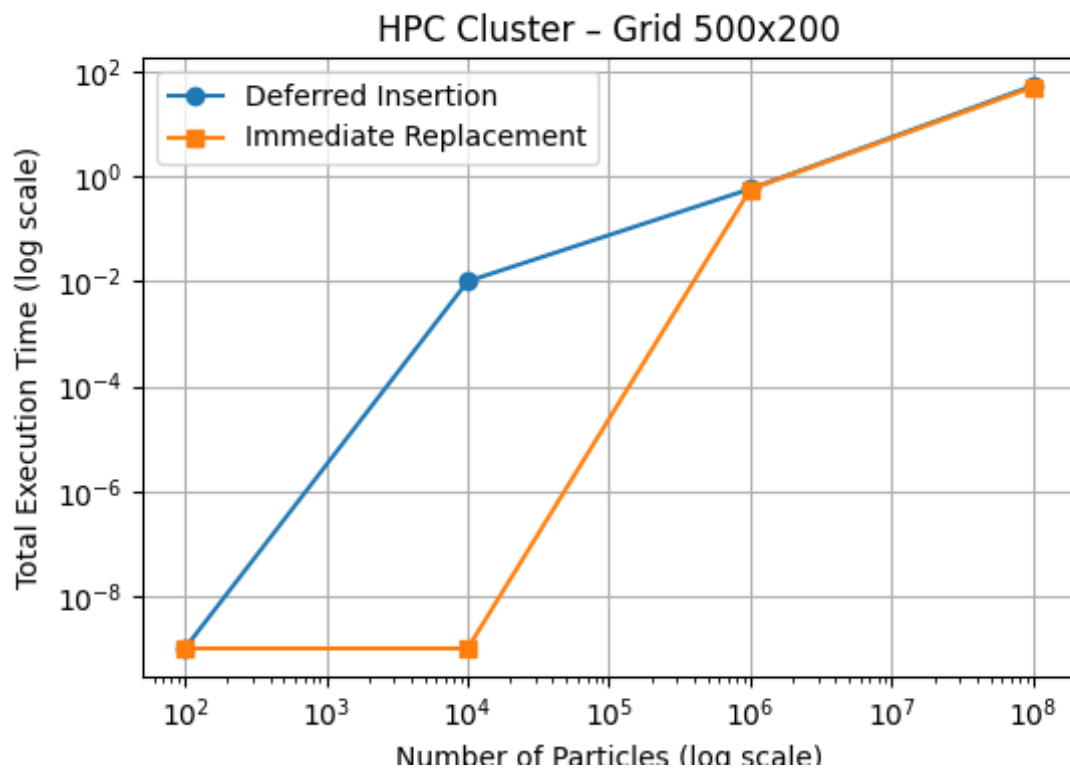
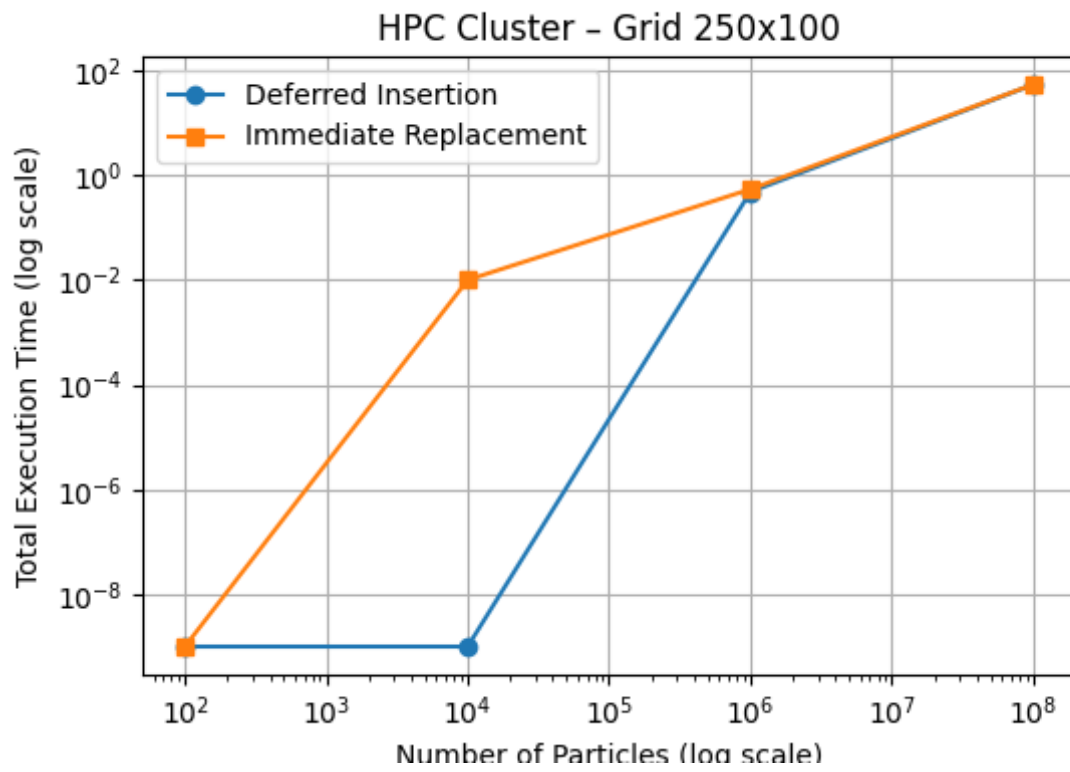
1. $N_x = 250$, $N_y = 100$
2. $N_x = 500$, $N_y = 200$
3. $N_x = 1000$, $N_y = 400$

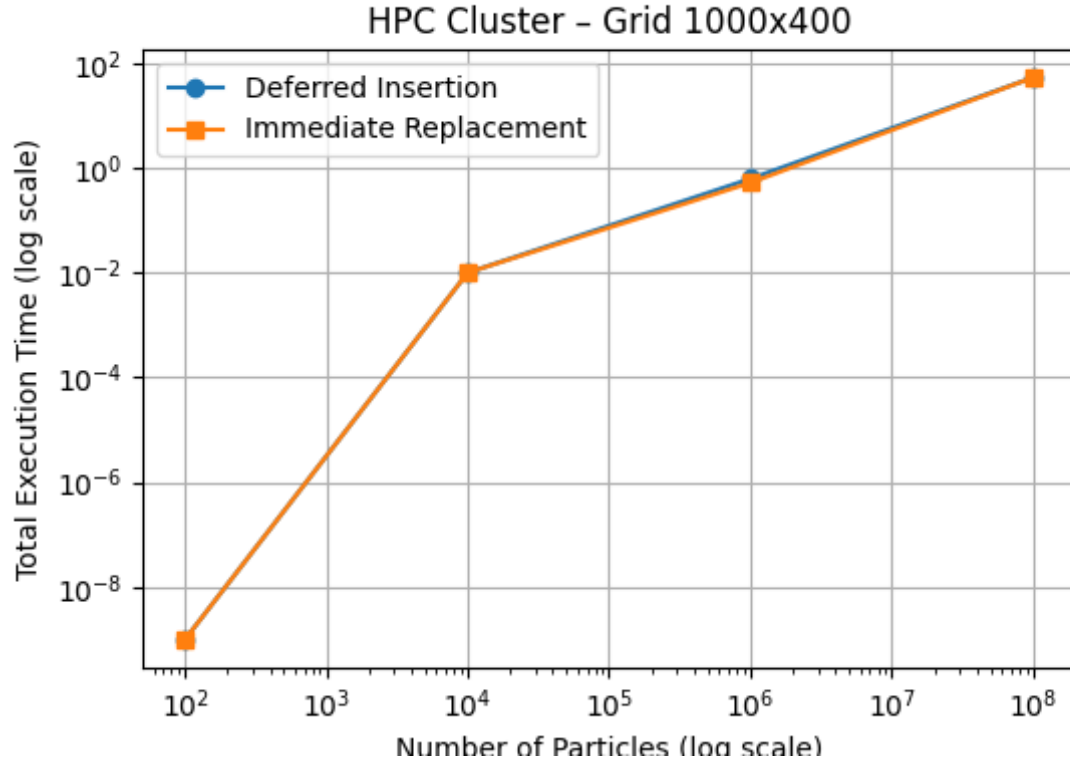
$$\text{Total execution time} = \sum_{\text{iter}=1}^{10} (T_{\text{interp}} + T_{\text{mover}}).$$

4.2 Results







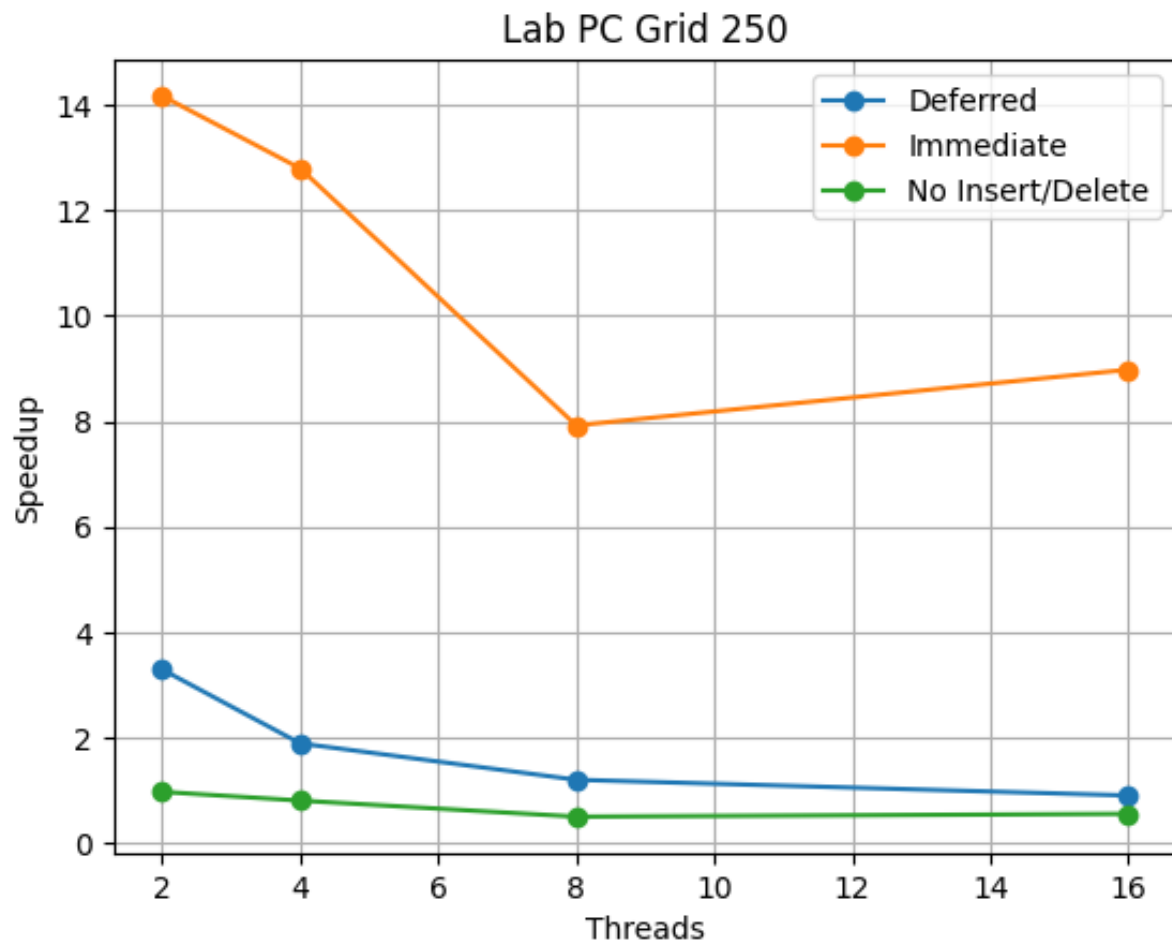


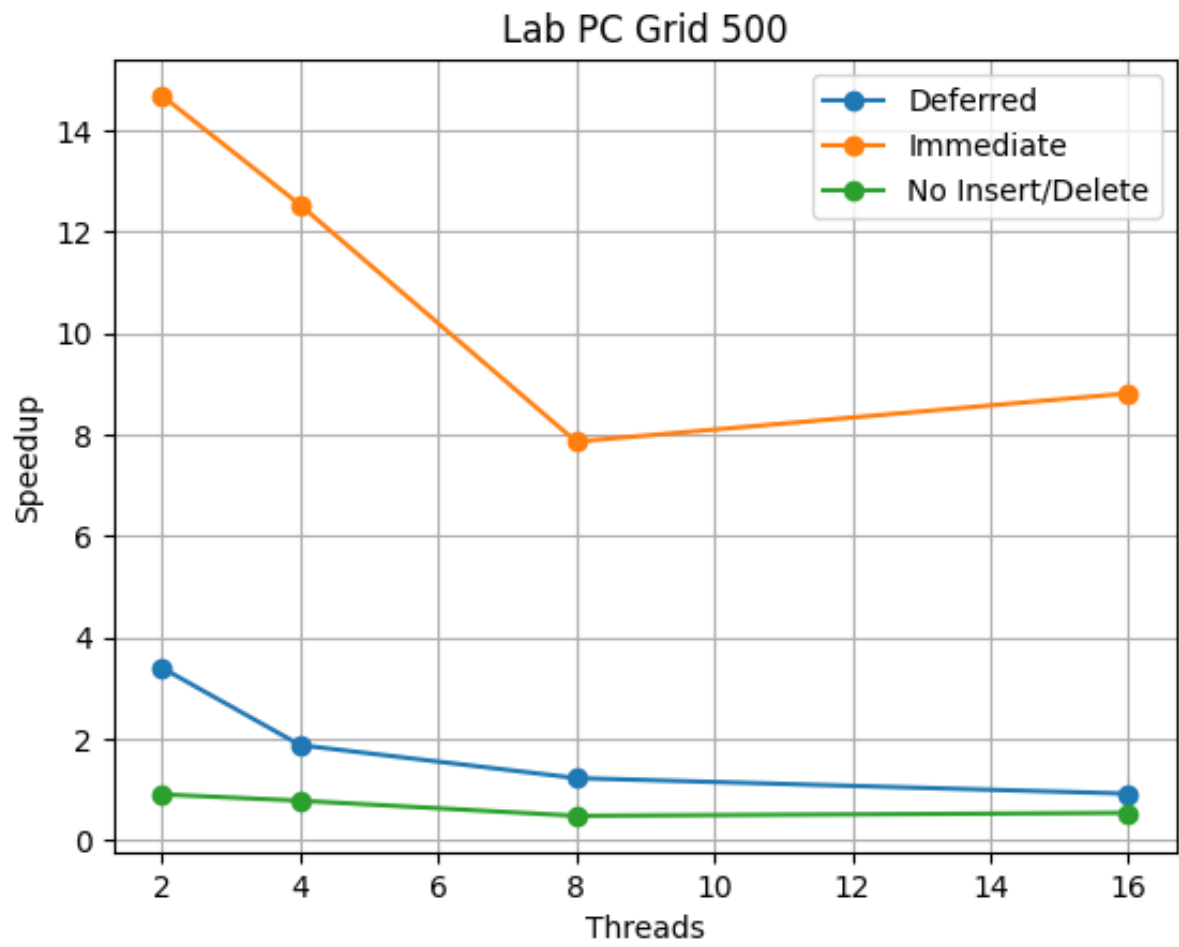
5 Experiment 02: Parallel Implementation and Scalability

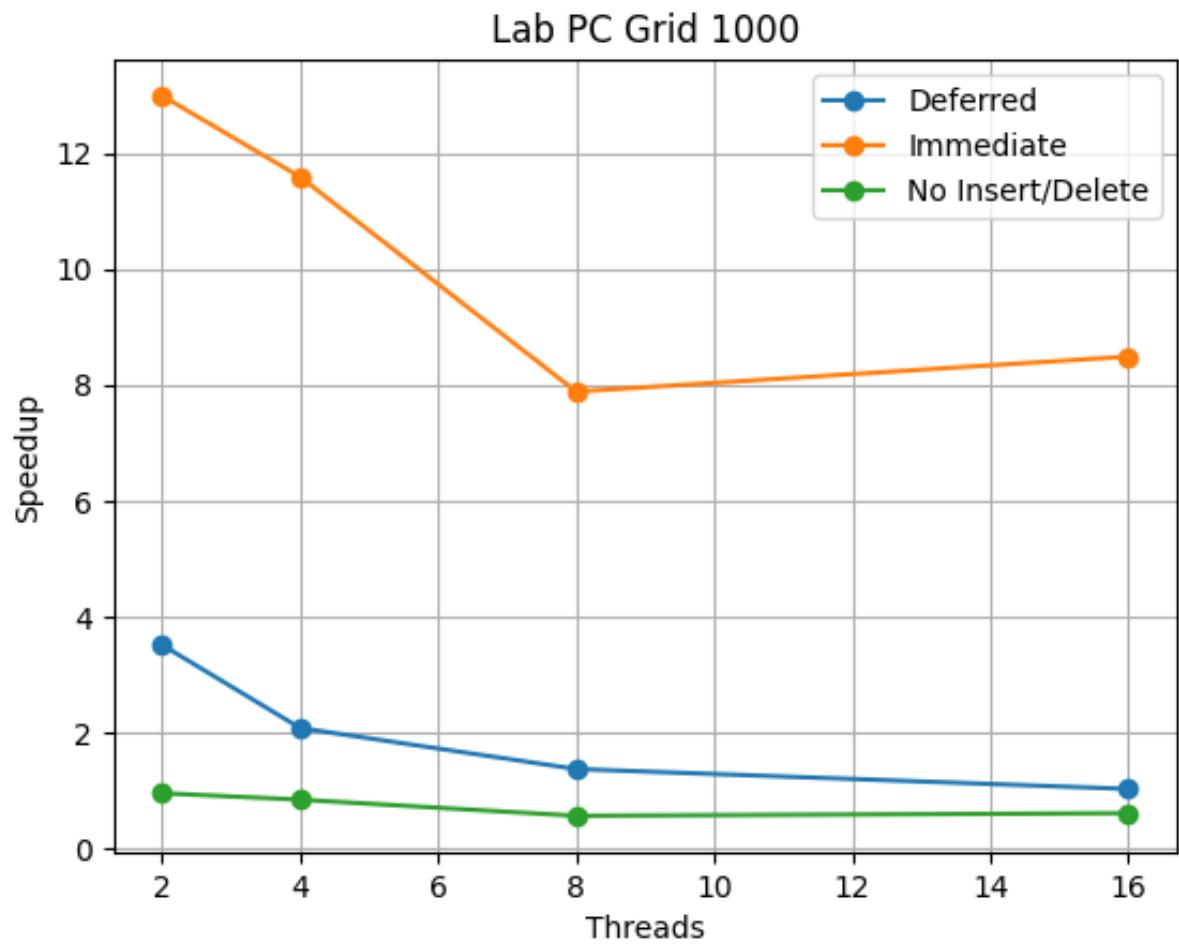
5.1 Setup

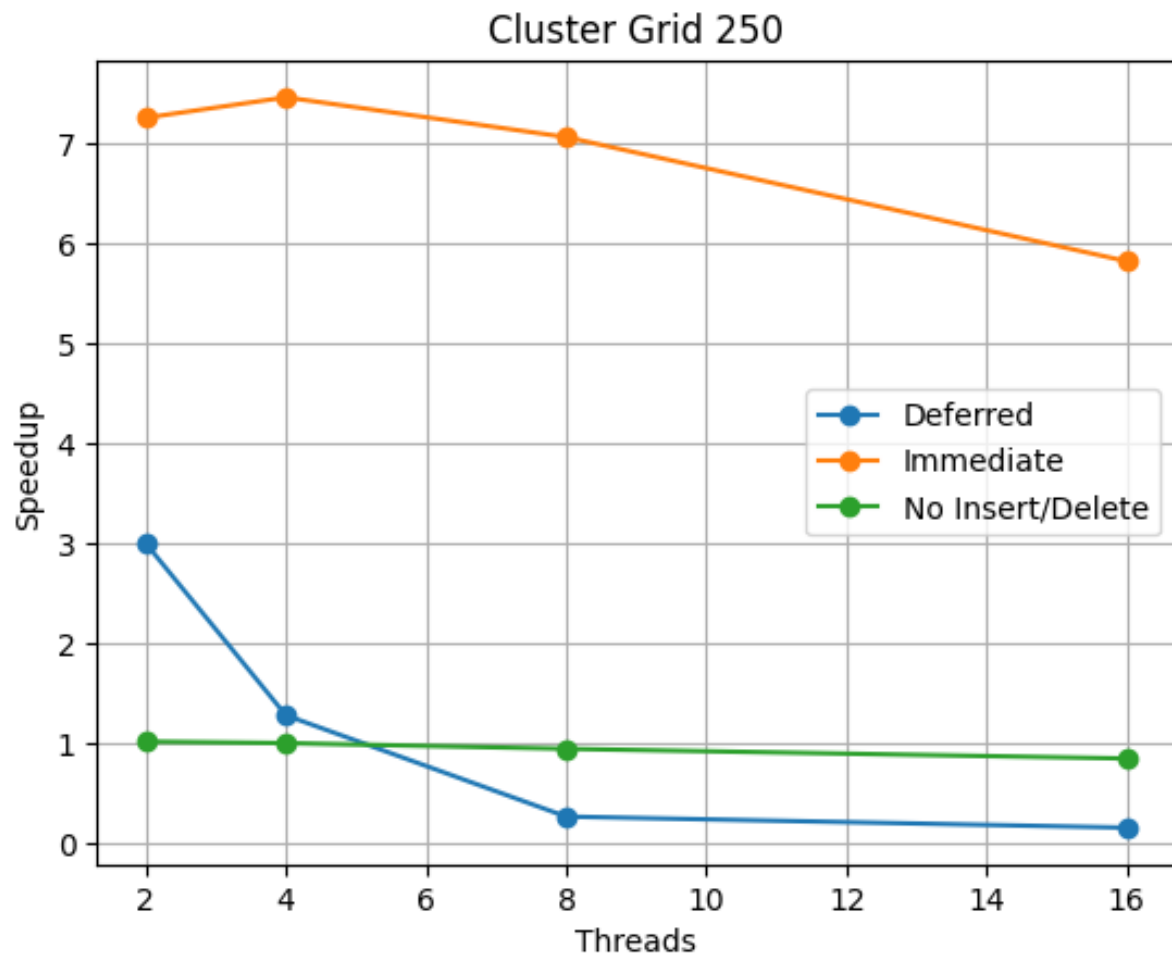
$N_p = 14 \times 10^6$, Maxiter = 10, thread counts: 2, 4, 8, 16. Speedup $S = T_{\text{serial}}/T_{\text{parallel}}$.

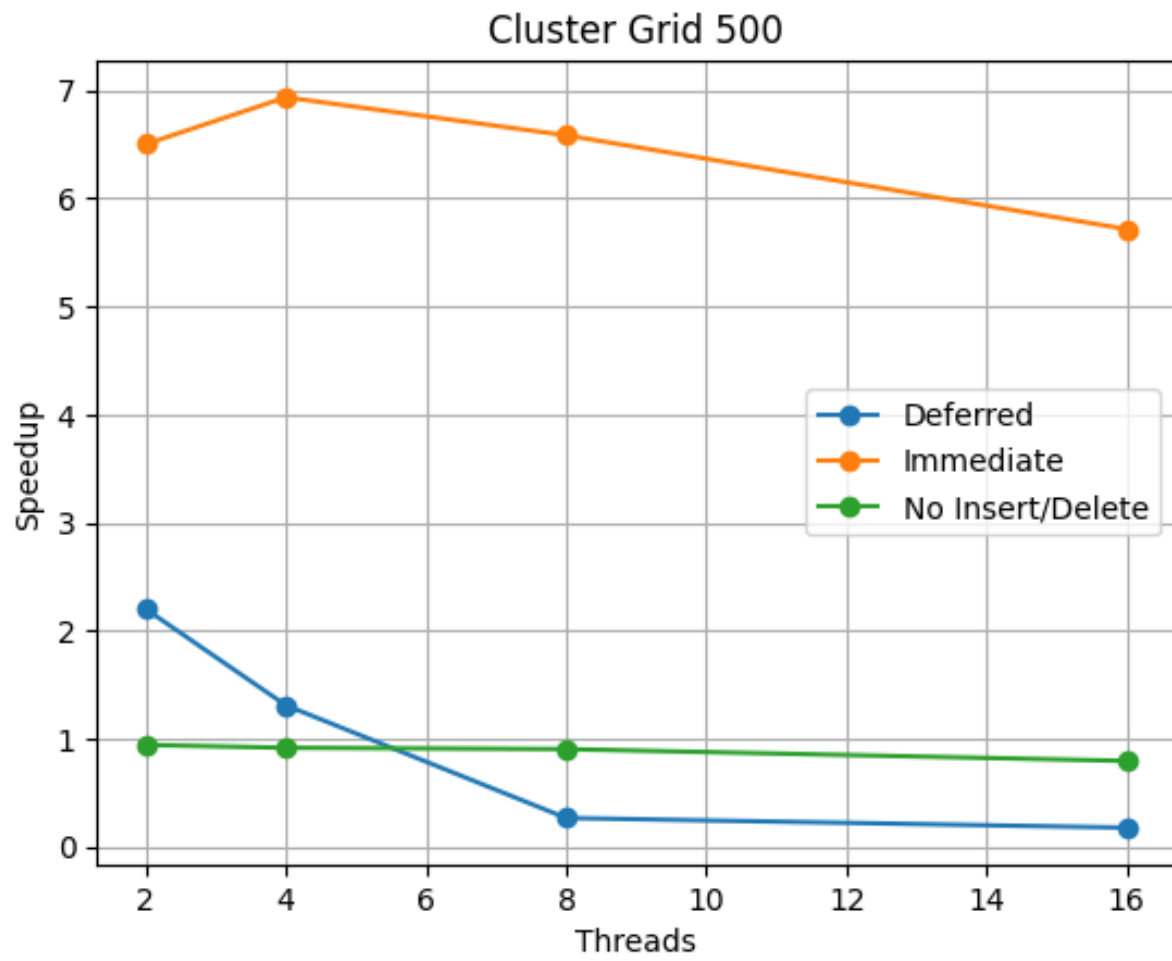
6 Results

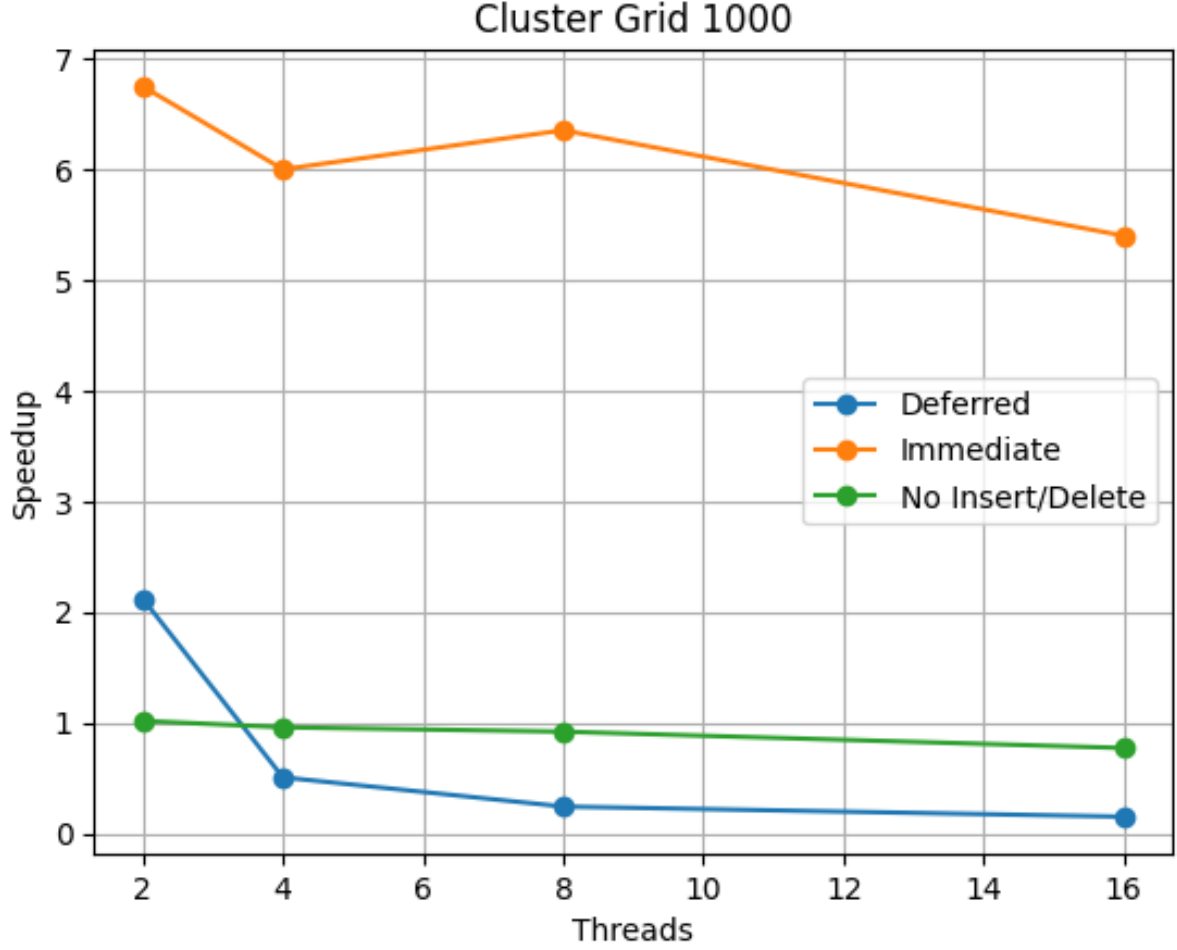












6.1 Lab PC vs. HPC Cluster Comparison

The Lab PC (Intel Core i5-12500) has a higher single-core clock speed and a larger L3 cache relative to the HPC node (Intel Xeon E5-2640 v3), which benefits small to medium particle counts. For very large N_p , the HPC cluster's dual-socket memory bandwidth advantage becomes apparent. *[Elaborate with measured data.]*

6.2 OpenMP Scalability and Effect of Insertion/Deletion

The basic Mover loop is embarrassingly parallel; however, insertion and deletion introduce **data dependencies**:

- *Deferred Insertion*: Counting deletions requires an atomic or reduction operation across threads. The compaction step (pushing voids to end) is sequential or requires careful synchronisation.
- *Immediate Replacement*: Each thread independently replaces out-of-domain particles, avoiding a global compaction step and enabling better scalability.

Measures taken to improve parallel scalability include:

- Using `#pragma omp parallel for with schedule(dynamic)` to handle load imbalance from variable deletion rates.

- Employing `reduction(+:del_count)` for the deletion counter.
- Thread-local random number generators to eliminate contention on a shared PRNG.

[Add quantitative speedup discussion referencing Figure ??.]

7 Bonus: Data Structures for Scalability

[Describe any alternative data structures explored, e.g. Structure-of-Arrays (SoA) vs Array-of-Structures (AoS), linked lists, ring buffers, or lock-free queues, and their effect on parallel performance.]

8 Conclusion

Both the Deferred Insertion and Immediate Replacement approaches were successfully implemented and parallelised with OpenMP. The Immediate Replacement Approach exhibited better parallel scalability because it avoids the global compaction step that serialises the Deferred approach. Memory bandwidth remains the primary performance bottleneck for large particle counts, consistent with the findings of Assignment 04.